

SMART FREIGHT

Cargo Damage & Incident Management System Comprehensive Technical Documentation & Implementation Reference

Version 1.0 | September 2026 | Multi-Tenant Logistics Platform | Steps 1–5 Fully Implemented

Table of Contents

- [Overview & Problem Statement](#1-overview--problem-statement)
- [Complete System Flow](#2-complete-system-flow)
- [Driver Dashboard Implementation](#3-driver-dashboard-implementation)
- [Incident Types Taxonomy](#4-incident-types-taxonomy)
- [Severity Classification Matrix](#5-severity-classification-matrix)
- [FastAPI Backend Architecture](#6-fastapi-backend-architecture)
- [Incident Creation Flow (Step-by-Step)](#7-incident-creation-flow-step-by-step)
- [Customer Dashboard & Notification Pipeline](#8-customer-dashboard--notification-pipeline)
- [Incident Status Lifecycle System](#9-incident-status-lifecycle-system)
- [Automatic Telemetry-Based Safety Detection](#10-automatic-telemetry-based-safety-detection)
- [Driver Alert & Confirmation Protocol](#11-driver-alert--confirmation-protocol)
- [API Data Flow & Endpoint Specifications](#12-api-data-flow--endpoint-specifications)
- [Security & Access Control Architecture](#13-security--access-control-architecture)
- [Photo Evidence & Storage Architecture](#14-photo-evidence--storage-architecture)
- [Error Handling & Edge Case Resilience](#15-error-handling--edge-case-resilience)
- [Files & Code Structure Index](#16-files--code-structure-index)
- [Technology Stack](#17-technology-stack)
- [Judge & Live Demonstration Scenario](#18-judge--live-demonstration-scenario)
- [Current Implementation Status Matrix](#19-current-implementation-status-matrix)
- [Current Limitations](#20-current-limitations)
- [Future Roadmap & Improvements](#21-future-roadmap--improvements)
- [System Architecture Diagrams](#22-system-architecture-diagrams)
- [Judge-Facing Executive Summary](#23-judge-facing-executive-summary)

1. Overview & Problem Statement

1.1 Context

In interstate multimodal freight transport—particularly across critical agricultural and industrial transit arteries such as the National Highway 16 (NH-16) corridor between Odisha (Bhubaneswar/Puri) and West Bengal (Kolkata/Howrah)—cargo is vulnerable to adverse in-transit events. These transit disruptions routinely lead to disputes, delayed insurance claims, cargo spoilage, and operational friction between freight carriers and cargo consignors/customers.

1.2 The Problem

During transit, cargo frequently experiences operational irregularities:

- **Physical Package Damage:** Dropped cartons, pallet shifts, torn shrink-wrapping, crushed lower containers due to shifting cargo loads.

- **Cold-Chain Temperature Violations:** Refrigeration unit compressor trips, door seal breaches, or prolonged defrost cycles causing perishable pharmaceuticals or fresh produce to exceed safe temperature envelopes (e.g., 2.0°C–8.0°C).
- **Leakage & Spillage:** Corrosive or liquid cargo punctures, barrel seepage, or fluid spillage threatening adjoining freight.
- **Accidents & Severe Impacts:** High-G decelerations, severe pothole strikes, or vehicular collisions damaging cargo bracing.
- **Unspecified Road Irregularities:** En-route security seal tampering, mechanical breakdowns, or unscheduled transshipment delays.

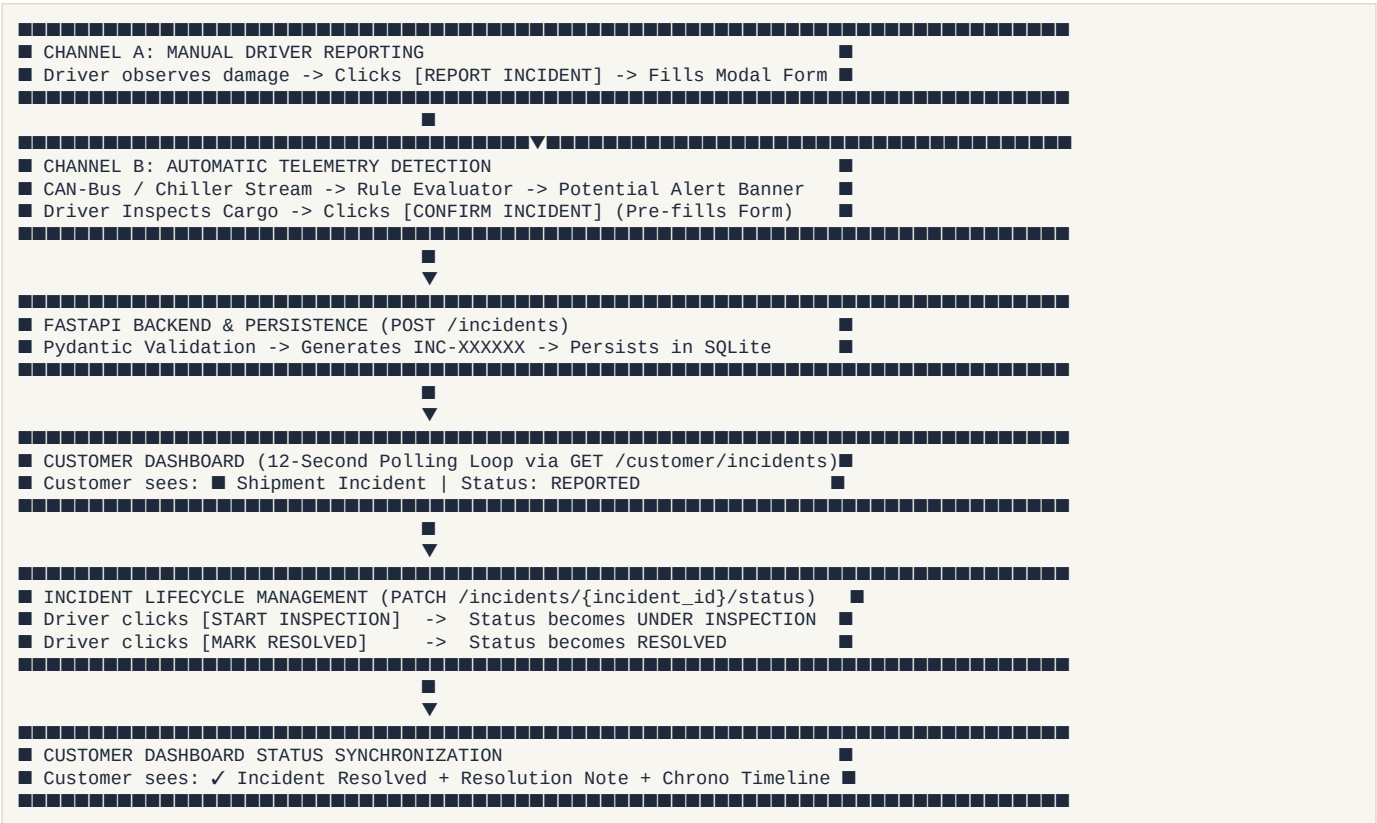
1.3 The Smart Freight Solution

The Smart Freight **Cargo Damage & Incident Management System** introduces a verified, closed-loop reporting and monitoring bridge between the active vehicle and the customer dashboard. Instead of relying on post-delivery arrival disputes or blind automated alarms, the platform enables:

1. **Immediate Manual Driver Reporting:** A dedicated, non-disruptive incident reporting modal inside the Driver Dashboard.
2. **Rule-Based Telemetry Safety Detection:** Continuous background evaluation of cold-chain temperature and accelerometer/impact telemetry against configurable thresholds.
3. **Human-in-the-Loop Driver Verification:** Flagging anomalies as *potential* incidents until the driver physically inspects the cargo and confirms findings.
4. **Relational Persistence:** Central storage of incidents and status lifecycles in SQLite with customer and shipment referential integrity.
5. **Real-Time Customer Transparency:** Automated polling-based delivery of incident alerts, chronological timelines, and driver resolution outcomes to the customer's dashboard.

2. Complete System Flow

The platform supports two complementary incident entry channels—Manual Reporting and Telemetry Detection—which converge into a unified backend lifecycle and customer notification pipeline.



3. Driver Dashboard Implementation

3.1 UI Placement & Interaction Points

The incident reporting capabilities are natively integrated into the **Current Trip Section** of the Driver Dashboard (frontend/src/app/driver/page.js):

- **Action Bar Button:** A prominent [REPORT INCIDENT] button situated in the active manifest header strip.

- **Secondary Quick Action:** A contextual [+ File New Incident] button located in the active incident strip.
- **Active Incident Manager:** A collapsible/expandable section displaying cards for all incidents recorded on the active manifest.

3.2 Incident Reporting Modal (CargoIncidentModal.js)

When triggered, a modal dialog renders with the following fields:

Field Name	Type	Description	Required	Default
shipment_id	String (read-only)	Auto-bound from active manifest (e.g., SF-E35749)	Yes	Active Manifest ID
incident_type	Select dropdown	Categorized incident classification	Yes	Package Damage
severity	Select pills	Urgency indicator (Low, Medium, High)	Yes	Medium
description	Textarea	Detailed observation notes entered by driver	Yes	Empty (or pre-filled via telemetry)
photo	File input (local)	Image capture from terminal camera or file picker	Optional	null
status	String (hidden)	Initial lifecycle state	Yes	REPORTED
timestamp	ISO timestamp	Client-generated time, validated by server	Yes	Current ISO time

3.3 Confirmation & Feedback

Upon submission:

1. The form disables and displays a loading spinner.
2. The payload is transmitted to POST /incidents.
3. If confirmed by the backend, the modal transitions to a success screen showing:
 - * **Incident ID:** Generated reference (e.g., INC-CCE70A).
 - * **Timestamp:** Converted to human-readable Indian Standard Time (IST).
 - * **Status:** REPORTED.
 - * **Customer Notification Confirmation:** Displays ✓ SENT TO CUSTOMER DASHBOARD.

4. Incident Types Taxonomy

The system enforces five standardized incident categories:

1. **Package Damage:** Outer carton tears, crushed crates, broken pallet straps, cargo displacement, or compromised physical packaging.
2. **Temperature Issue:** Chiller temperature excursions outside required thermal envelopes for cold-chain shipments.
3. **Leakage / Spillage:** Liquid cargo seepage, barrel puncture, chemical drippage, or condensation pooling.
4. **Accident / Impact:** Vehicular collision, sudden emergency deceleration, or heavy pothole/road shocks affecting cargo alignment.
5. **Other:** Irregularities not covered by standard classifications (e.g., seal tampering, customs checkpoint inspection delays).

5. Severity Classification Matrix

Severity	Color Coding	Operational Definition	Telemetry Trigger Rules
LOW	Forest Green (#2D5926 / #EBF3EA)	Minor cosmetic irregularity; low probability of cargo loss.	Minor temp deviation (≤ 2.0°C excursion) or minor shock (1.8g-2.4g).
MEDIUM	Amber Gold (#B8711E / #FEF6E8)	Notable operational breach; physical cargo check required.	Persistent temp breach (2.1°C-8.0°C excursion for ≥ 8s), moderate shock (2.5g-3.7g), or harsh braking (-4.5 to -6.5 m/s²).
HIGH	Rust Terracotta (#BA4336 / #FDF0EA)	Critical breach; high risk of total batch spoilage or cargo destruction.	Extreme temp excursion (> 8.0°C breach), violent impact (≥ 3.8g), or emergency braking (< -6.5 m/s²).

6. FastAPI Backend Architecture

6.1 Core Modules & Files

- main.py & backend/main.py: Core FastAPI application exposing REST routes, CORS middleware, JWT decoding, and dependency injection.
- database.py: SQLite abstraction layer handling connection pooling, schema initialization, table indices, and relational queries.
- telemetry_service.py: Pure rule evaluation engine containing threshold matrices and potential anomaly evaluation logic.
- auth.py: Cryptographic HS256 JWT access token generation and signature verification.

6.2 Relational Database Schema (smart_freight.db)

Table: cargo_incidents

```
CREATE TABLE IF NOT EXISTS cargo_incidents (  
  id TEXT PRIMARY KEY,  
  shipment_id TEXT NOT NULL,  
  driver_id TEXT DEFAULT NULL,  
  incident_type TEXT NOT NULL,  
  severity TEXT NOT NULL,  
  description TEXT NOT NULL,  
  photo_data TEXT DEFAULT NULL,  
  photo_name TEXT DEFAULT NULL,  
  timestamp TEXT NOT NULL,  
  status TEXT DEFAULT 'REPORTED',  
  created_at TEXT NOT NULL,  
  resolution_note TEXT DEFAULT NULL,  
  inspected_at TEXT DEFAULT NULL,  
  resolved_at TEXT DEFAULT NULL,  
  FOREIGN KEY(shipment_id) REFERENCES shipments(id)  
);  
CREATE INDEX IF NOT EXISTS idx_incidents_shipment_id ON cargo_incidents(shipment_id);
```

Table: dismissed_telemetry_events

```
CREATE TABLE IF NOT EXISTS dismissed_telemetry_events (  
  id TEXT PRIMARY KEY,  
  shipment_id TEXT NOT NULL,  
  driver_id TEXT DEFAULT NULL,  
  rule_type TEXT NOT NULL,  
  telemetry_summary TEXT NOT NULL,  
  dismissed_at TEXT NOT NULL  
);  
CREATE INDEX IF NOT EXISTS idx_dismissed_shipment_rule  
ON dismissed_telemetry_events(shipment_id, rule_type);
```

7. Incident Creation Flow (Step-by-Step)

- 1. Trigger:** Driver clicks [REPORT INCIDENT] in the driver terminal.
- 2. Auto-Association:** Active manifest ID (shipment_id) is injected into the modal state.
- 3. Driver Input:** Driver selects incident type, severity, and enters description notes.
- 4. Submission:** Frontend executes submitCargoIncident() transmitting JSON to POST /incidents.
- 5. Authentication Check:** Backend checks Bearer JWT header (if provided) to extract caller identity.
- 6. Schema Validation:** FastAPI validates fields against IncidentCreateRequest (severity must be Low, Medium, or High).
- 7. Shipment Existence:** Backend verifies shipment exists in SQLite; returns 404 Not Found if missing.
- 8. ID Generation:** A unique identifier is generated with prefix INC- and 6 hexadecimal characters (e.g., INC-4BBB54).
- 9. Persistence:** Record inserted into cargo_incidents table with status = 'REPORTED'.
- 10. Response:** HTTP 201 Created returned with full IncidentResponse payload.
- 11. Driver Confirmation:** Modal renders success view confirming Customer notification: SENT TO CUSTOMER DASHBOARD.
- 12. Customer Visibility:** The customer's polling loop fetches the incident within the next 12-second cycle.

8. Customer Dashboard & Notification Pipeline

8.1 Retrieval Architecture

The Customer Dashboard does not query raw incidents directly by arbitrary ID. Instead:

- GET /customer/incidents is invoked with the customer's Bearer JWT.
- Backend inspects the token sub (e.g., USR-DEMO-001).
- Executes relational join:

```
```sql  
SELECT i.*, s.product_type, s.pickup_location, s.destination, s.user_id AS customer_id
FROM cargo_incidents i
JOIN shipments s ON i.shipment_id = s.id
WHERE s.user_id = ?
ORDER BY i.created_at DESC;
```
```

4. Only incidents belonging to shipments owned by the authenticated customer are returned.

8.2 Customer Notification Component (CustomerIncidentNotification.js)

Mounted in frontend/src/app/shipments/page.js and frontend/src/app/page.js.

- **Polling Loop:** Executes every **12 seconds** using setInterval.
- **Lifecycle Cleanup:** clearInterval is executed upon component unmount to prevent memory leaks.
- **Notification Banner:** Displays at top of dashboard when active incidents exist:

* ■ **Shipment Incident**

* Manifest ID, Incident Type, Severity badge, and Reported Time (IST).

* Contextual customer message:

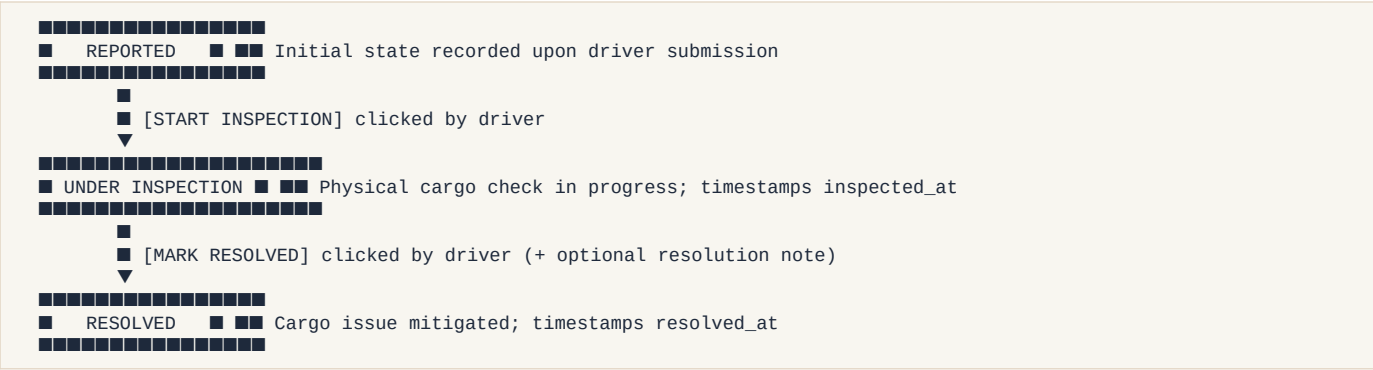
REPORTED: "Your driver has reported an issue with this shipment."*

UNDER INSPECTION: "Your driver is currently inspecting the shipment."*

RESOLVED: "The reported shipment issue has been resolved."* (with ✓ Incident Resolved header).

- **View Details Modal:** Opens full inspection view with multi-step chronological timeline.

9. Incident Status Lifecycle System



9.1 Status Definitions

- **REPORTED:** Incident has been formally entered into the ledger. Operations and customer are notified that an irregularity exists.
- **UNDER INSPECTION:** Driver has halted the vehicle or entered the cargo compartment to perform physical inspection. Timestamps inspected_at.
- **RESOLVED:** Driver has addressed the issue (e.g., restrapped pallet, resealed container, recalibrated chiller). Timestamps resolved_at and saves resolution note.

9.2 Driver Incident Card Controls (DriverIncidentCard.js)

Renders on the Driver Dashboard for each manifest incident:

- For REPORTED: Displays [START INSPECTION] button.
- For UNDER INSPECTION: Displays [MARK RESOLVED] button which prompts an optional resolution note modal before confirming.
- For RESOLVED: Displays green verified status with resolution notes.

10. Automatic Telemetry-Based Safety Detection

10.1 Safety Principle: Potential vs. Actual Damage

CRITICAL ARCHITECTURAL RULE:

Telemetry detection triggers an alert for a **"POTENTIAL INCIDENT"**. It does **NOT** automatically declare cargo damaged, nor does it create a customer incident without human driver verification.

10.2 Configurable Rule Thresholds (telemetry_service.py)

```
TELEMETRY_CONFIG = {
    "temperature": {
        "reefer_min_c": 2.0,
        "reefer_max_c": 8.0,
        "low_breach_delta_c": 2.0,      # 8.1°C to 10.0°C -> LOW
        "medium_breach_delta_c": 5.0,   # 10.1°C to 13.0°C -> MEDIUM
        "high_breach_delta_c": 8.0,     # > 16.0°C -> HIGH
        "persistence_seconds": 8,       # Must persist for >= 8s to prevent transient false alarms
    },
    "impact": {
        "low_g": 1.8,                  # 1.8g to 2.4g -> LOW (minor road shock)
        "medium_g": 2.5,               # 2.5g to 3.7g -> MEDIUM (substantial shock)
        "high_g": 3.8,                 # >= 3.8g -> HIGH (severe shock / collision)
    },
    "braking": {
        "medium_decel_mps2": -4.5,     # -4.5 to -6.5 m/s² -> MEDIUM (harsh braking)
        "high_decel_mps2": -6.5,       # < -6.5 m/s² -> HIGH (emergency stop)
    },
    "vibration": {
        "enabled": False,
        "reason": "Hardware vibration sensors not present in current fleet specification.",
    },
}
```

10.3 Duplicate Alert Suppression & Dismissals

When a driver clicks [DISMISS]:

- 1. POST /telemetry/dismiss records (shipment_id, rule_type) in dismissed_telemetry_events.
- 2. Subsequent evaluations for that same shipment and rule return triggered: False, dismissed: True.
- 3. The driver is not repeatedly harassed by alerts for the same acknowledged event.

11. Driver Alert & Confirmation Protocol

When anomalous telemetry is detected, [DriverTelemetryAlert.js](file:///C:/Users/ABHISEK/.gemini/antigravity/scratch/smart-fr eight/frontend/src/components/DriverTelemetryAlert.js) renders a non-blocking banner on the Driver Dashboard:

```
■ POTENTIAL INCIDENT DETECTED
Temperature has exceeded the safe range for this shipment.
Shipment: SF-E35749 • Vehicle: Refrigerated Van • Detected: 14:32 IST
[ INSPECT ] [ CONFIRM INCIDENT ] [ DISMISS ]
```

- [INSPECT]: Expands sensor diagnostics showing current reading vs. threshold.
- [CONFIRM INCIDENT]: Opens CargoIncidentModal pre-filled with incident type, severity, description, and source tag AUTOMATIC_TELEMETRY. Driver can edit prior to submission.
- [DISMISS]: Clears alert, records dismissal in backend, and sends **zero** notifications to customer.

12. API Data Flow & Endpoint Specifications

12.1 Endpoint Summary

| Method | Endpoint | Purpose | Auth Required | Request Body | Status Code |
|--------|---------------------------------|---|----------------------------|-----------------------------|--------------|
| POST | /incidents | Submit & persist cargo incident | Optional (extracts driver) | IncidentCreateRequest | 201 Created |
| GET | /incidents/{shipment_id} | Fetch all incidents for shipment | Yes (Driver or Consignor) | None | 200 OK / 403 |
| GET | /incidents/{shipment_id}/latest | Fetch most recent incident | Yes (Driver or Consignor) | None | 200 OK / 404 |
| GET | /customer/incidents | Fetch all incidents owned by customer | Yes (Bearer Token) | None | 200 OK |
| PATCH | /incidents/{incident_id}/status | Update incident lifecycle status | Optional (checks role) | IncidentStatusUpdateRequest | 200 OK / 422 |
| POST | /telemetry/evaluate | Evaluate telemetry stream against rules | No | TelemetryEvaluationRequest | 200 OK |
| POST | /telemetry/dismiss | Dismiss potential telemetry alert | Optional (extracts driver) | TelemetryDismissRequest | 200 OK |

12.2 Request/Response Schemas

POST /incidents Payload

```
{
  "shipment_id": "SF-E35749",
  "incident_type": "Package Damage",
  "severity": "High",
  "description": "Severe impact observed on pallet 3; crate cracked.",
  "status": "REPORTED",
  "photo_name": "high_damage.jpg"
}
```

PATCH /incidents/{incident_id}/status **Payload**

```
{
  "status": "RESOLVED",
  "resolution_note": "Damaged outer packaging inspected. Internal cargo unaffected and resealed."
}
```

13. Security & Access Control Architecture

- 1. **Authentication:** Uses cryptographically signed HS256 JWT tokens. Users receive tokens upon login with claims: sub (User ID), role (consumer or driver), email, and name.
- 2. **Multi-Tenant Ownership Isolation:**
 - * Consignors querying GET /customer/incidents only receive incidents for shipments matching shipments.user_id == token.sub.
 - * If Consignor A requests GET /incidents/{shipment_id} for Consignor B's shipment, backend responds with HTTP 403 Forbidden.
- 3. **Current MVP Limitation:** While token decoding and ownership checks are implemented on incident routes, the local development server permits demo access tokens for testing. Production deployment will require strict OAuth2 bearer token expiration enforcement.

14. Photo Evidence & Storage Architecture

- **Current Implementation Status:** PARTIALLY IMPLEMENTED (DEMO STORAGE)
- **Capture Behavior:** Drivers can select or capture an image via file picker in CargoIncidentModal.
- **Client Handling:** The browser extracts filename and generates a local data:image/... preview URI.
- **Server Storage:** The backend stores photo_name and optional preview metadata in SQLite.
- **Honest Limitation:** Full binary blobs or cloud object storage (e.g., AWS S3 / Cloudinary) are **not** yet connected. The system does **not** generate fake URLs; if an image URL is not genuinely present, it indicates "Photo recorded by driver: {filename} (local terminal capture)".

15. Error Handling & Edge Case Resilience

Scenario | Server Response / Client Handling

Invalid severity (e.g., "CRITICAL") | Backend rejects with 422 Unprocessable Content.

Invalid lifecycle status (e.g., "COMPLETED") | Backend rejects with 422 Unprocessable Content listing valid options.

Querying non-existent incident ID | Backend returns 404 Not Found.

Consignor querying foreign shipment | Backend returns 403 Forbidden with access denied detail.

Backend offline during telemetry check | Frontend falls back to client-side evaluation without crashing.

Polling unmount | Component cleans up intervals preventing memory leaks.

16. Files & Code Structure Index

| Component | File Path | Purpose |
|-----------------------|---|---|
| Backend API | main.py | Primary FastAPI application with incident, telemetry, and status routes. |
| Backend API (Backup) | backend/main.py | Secondary application entry point maintained in sync. |
| Database Layer | database.py | SQLite connection, table schemas, migrations, and relational queries. |
| Rule Engine | telemetry_service.py | Rule-based evaluator for temperature, impact, and deceleration. |
| Auth Service | auth.py | JWT token issuance, password hashing, and token verification. |
| Driver Dashboard | frontend/src/app/driver/page.js | Driver terminal page hosting active trip, maps, telemetry, and incidents. |
| Customer Dashboard | frontend/src/app/shipments/page.js | Customer shipment ledger hosting incident banners and inspector. |
| Driver Incident Modal | frontend/src/components/CargoIncidentModal.js | Modal dialog for manual filing and telemetry confirmation. |

Driver Incident Card | frontend/src/components/DriverIncidentCard.js | Driver lifecycle card with [START INSPECTION] and [MARK RESOLVED].

Driver Alert Banner | frontend/src/components/DriverTelemetryAlert.js | Potential incident alert banner with [INSPECT], [CONFIRM], [DISMISS].

Telemetry Simulator | frontend/src/components/DriverTelemetrySimulator.js | Judge/demo testing cockpit simulating normal, breach, impact, and braking.

Customer Notification | frontend/src/components/CustomerIncidentNotification.js | Customer-facing banner, polling manager, and lifecycle timeline modal.

Incident Client Service | frontend/src/services/incidentService.js | Frontend API client for incident reporting, querying, and status updates.

Telemetry Client Service | frontend/src/services/telemetryService.js | Frontend API client for telemetry evaluation and alert dismissals.

17. Technology Stack

- **Frontend Framework:** Next.js 16.3.1 (React 19, Turbopack, App Router)
- **Styling & Design System:** Tailwind CSS with custom Smart Freight color palette (Terracotta, Parchment, Amber, Forest)
- **Iconography:** Lucide React
- **Backend Framework:** Python 3.13 / FastAPI (Async ASGI)
- **ASGI Server:** Uvicorn
- **Database:** SQLite 3 (smart_freight.db) with relational foreign keys and indices
- **Security & Tokens:** PyJWT / HMAC-SHA256
- **Data Validation:** Pydantic v2
- **Mapping:** Leaflet / OpenStreetMap / Open Source Routing Machine (OSRM)

18. Judge & Live Demonstration Scenario

To demonstrate the full incident lifecycle to evaluation judges:

1. **Start System:** Open Driver Dashboard at `http://localhost:3000/driver` and Customer Dashboard at `http://localhost:3000/shipments` in side-by-side browser windows.
2. **Observe Baseline:** Driver Dashboard shows active manifest SF-E35749 with normal telemetry (4.2°C, 0.2g). No alerts are present.
3. **Simulate Telemetry Breach:** Expand the **Demo Test Harness** in the Driver Dashboard. Click ■ Temp Breach (sets 10.8°C for 12s).
4. **Driver Alert Triggers:** Driver Dashboard displays ■ POTENTIAL INCIDENT DETECTED. Emphasize that **no notification has been sent to the customer yet**.
5. **Inspect Diagnostics:** Driver clicks [INSPECT] to view the exact sensor telemetry breakdown.
6. **Driver Confirmation:** Driver clicks [CONFIRM INCIDENT]. The reporting modal opens with pre-filled telemetry data. Driver reviews and clicks Confirm & Submit Report.
7. **Customer Receives Notification:** Within 12 seconds, the Customer Dashboard displays: ■ Shipment Incident | Status: REPORTED.
8. **Inspection Transition:** Driver clicks [START INSPECTION]. The Customer Dashboard updates to: Status: UNDER INSPECTION ("*Your driver is currently inspecting the shipment*").
9. **Resolution Transition:** Driver clicks [MARK RESOLVED], enters note: "*Packaging resealed. Internal goods verified intact*", and confirms.
10. **Customer Sees Resolved:** Customer Dashboard updates to green: ✓ Incident Resolved | Status: RESOLVED. Clicking [View Details] reveals the complete 3-step chronological timeline and the driver's note.

19. Current Implementation Status Matrix

| Feature Module | Status | Technical Implementation Notes |
|----------------------------|-------------|---|
| Manual Incident Reporting | IMPLEMENTED | Full modal UI, input validation, and SQLite persistence. |
| Incident REST API | IMPLEMENTED | Endpoints for creation, retrieval, latest query, and status updates. |
| Relational Persistence | IMPLEMENTED | SQLite cargo_incidents with indexes and foreign key references. |
| Driver Incident Management | IMPLEMENTED | Cards with [START INSPECTION] and [MARK RESOLVED] controls. |
| Customer Notifications | IMPLEMENTED | 12-second polling with dynamic messages and green resolved state. |
| Chronological Timeline | IMPLEMENTED | Records timestamp, inspected_at, resolved_at, and resolution note. |
| Telemetry Safety Rules | IMPLEMENTED | Evaluates temperature envelope persistence, impact G-force, and decel. |
| Driver Telemetry Alerts | IMPLEMENTED | Displays <i>Potential Incident</i> banner with Inspect, Confirm, Dismiss. |

Duplicate Alert Suppression | **IMPLEMENTED** | Dismissed alerts stored in dismissed_telemetry_events.

Telemetry Demo Harness | **SIMULATED** | Safe UI simulation cockpit feeding into real detection pipeline.

Multi-Tenant Access Control | **IMPLEMENTED** | JWT-based ownership verification returning HTTP 403 on cross-access.

Photo Evidence | **PARTIALLY IMPLEMENTED** | Client capture and filename logged; remote cloud object store deferred.

Real-Time WebSockets | **NOT IMPLEMENTED** | System utilizes reliable 12-second polling instead of WebSockets.

Machine Learning Risk Model | **NOT IMPLEMENTED** | Deferred to subsequent development phases per instructions.

20. Current Limitations

- Telemetry Ingestion:** Real physical CAN-Bus/OBD-II and Bluetooth BLE container sensors are simulated via the Driver Cockpit test harness.
- Photo Storage:** Photos are previewed locally and referenced by filename on the server; persistent cloud CDN storage (e.g., S3) is not yet active.
- External Notifications:** SMS, WhatsApp, and push notifications are deferred; updates are delivered in-app.
- Communication Protocol:** Updates rely on 12-second HTTP polling rather than persistent bi-directional WebSocket channels.
- Predictive ML:** The current engine uses deterministic, rule-based policies rather than predictive machine learning models.

21. Future Roadmap & Improvements

- Physical IoT Sensor Integration:** Hardware pairing with BLE temperature data loggers and vehicle telematics units (VTUs).
- WebSocket Event Gateway:** Transitioning customer notification polling to low-latency Server-Sent Events (SSE) or WebSockets.
- Cloud Object Store:** Connecting S3 or Cloudinary for encrypted in-transit cargo photographic evidence.
- Automated Omnichannel Alerts:** Automated SMS / WhatsApp status updates via Twilio for consignees without open browser sessions.
- Predictive Cargo Risk ML:** Training machine learning models on NH-16 road topography and chiller telemetry to predict cargo degradation before breaches occur.

22. System Architecture Diagrams

Diagram 1: Complete Operational Flow

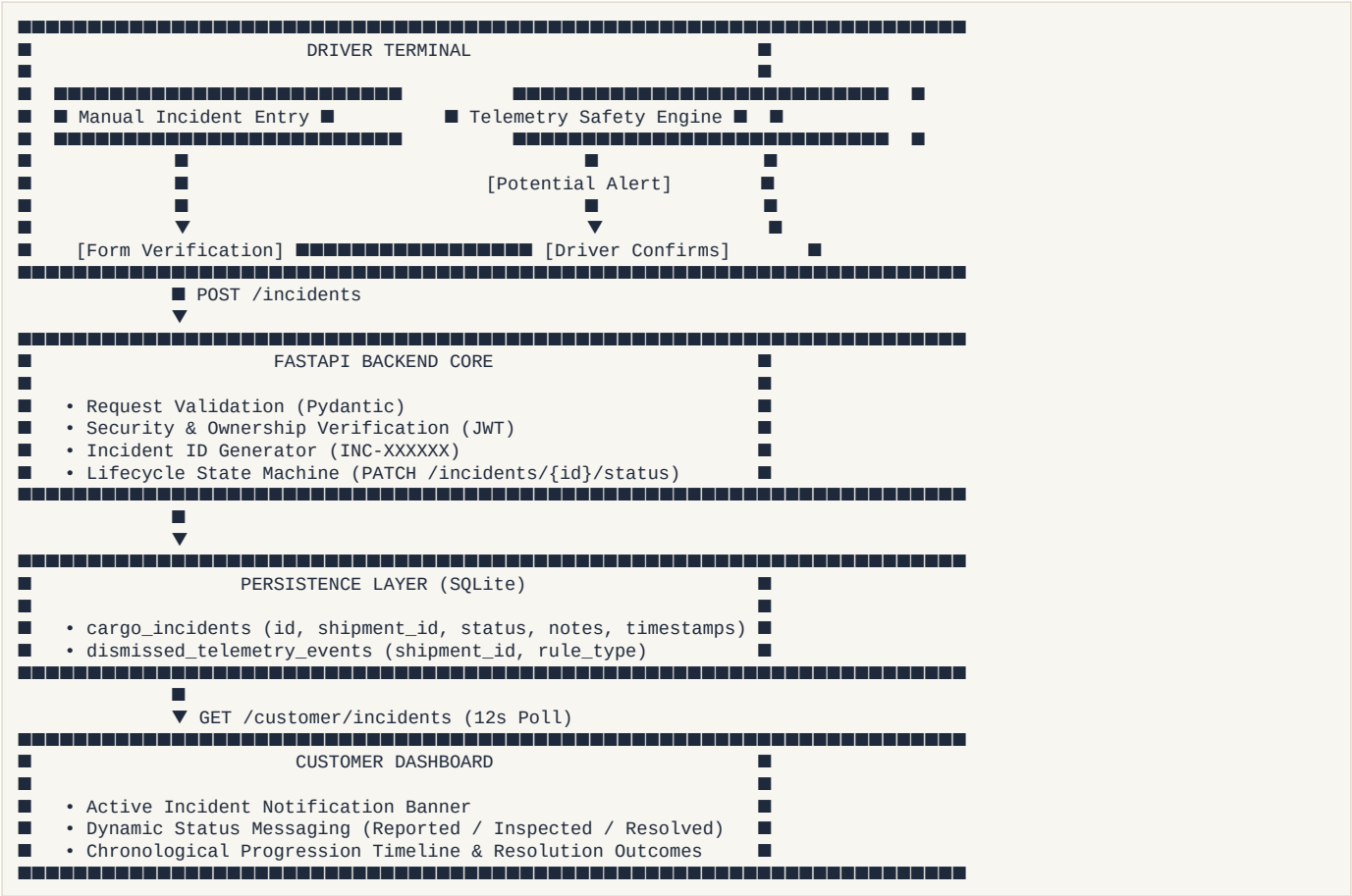
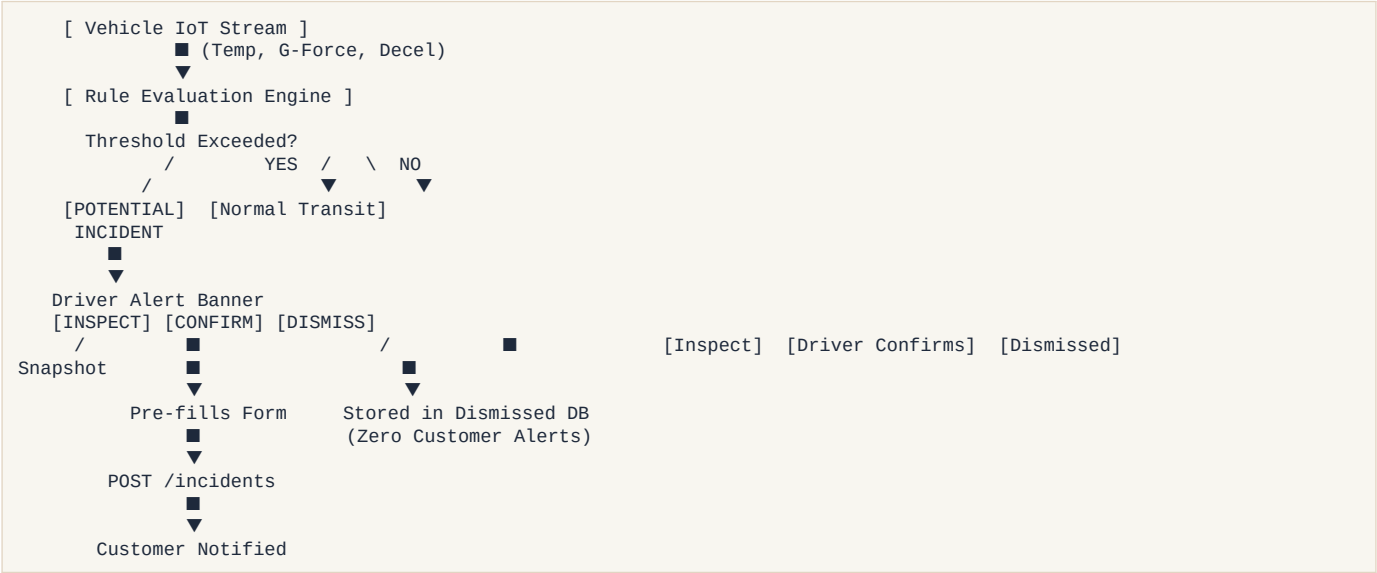


Diagram 2: Telemetry Safety & Human-in-the-Loop Protocol



23. Judge-Facing Executive Summary

"The Smart Freight Cargo Damage & Incident Management System does not simply detect damage and blindly notify the customer. It identifies potential abnormal events through real-time telemetry rules, provides the driver with an immediate opportunity to physically inspect and verify the cargo condition, records the verified incident through a secured FastAPI backend ledger, and delivers transparent, synchronized updates to the customer dashboard. The customer can follow the incident's complete lifecycle—from initial report to physical inspection and final resolution—ensuring operational accountability, reducing transit disputes, and building trust across the freight supply chain."